LAPORAN PROYEK INTERNET OF THINGS

Sistem Deteksi Kebocoran Gas LPG dengan Penutup Otomatis Tuas Regulator Berbasis ESP32 dan Pemantauan MQTT

Disusun untuk Study Case Selection Phase 2:
IoT Device and Hardware Discovery
Samsung Innovation Campus Batch 8



Disusun oleh:

Nama tim: Malas Mikir Nama

Frenaldy Bestabba Hasugian	(5025241156)
Muhammad Sayyid Tsabit	(5024241013)
Nicholas	(5022241065)
Mitra Partogi	(5025241017)

Institut Teknologi Sepuluh Nopember

2026

KATA PENGANTAR

Puji syukur kami panjatkan kepada Tuhan Yang Maha Esa karena laporan proyek Internet of Things berjudul *Sistem Deteksi Kebocoran Gas LPG dengan Penutup Otomatis Tuas Regulator Berbasis ESP32 dan Pemantauan MQTT* dapat diselesaikan. Laporan ini disusun untuk memenuhi Study Case Selection Phase 2: IoT Device and Hardware Discovery pada program Samsung Innovation Campus Batch 8.

Laporan ini menjelaskan proses ideasi, pemilihan perangkat, perancangan rangkaian, pemrograman ESP32, komunikasi MQTT, visualisasi data, hingga pengujian sistem pada simulator Wokwi. Kami menyadari laporan ini masih memiliki kekurangan, sehingga kritik dan saran yang membangun sangat kami harapkan.

Surabaya, 30 Oktober 2026

Tim Malas Mikir Nama

DAFTAR ISI

KATA PENGANTAR **ii**

DAFTAR ISI **iii**

DAFTAR GAMBAR **v**

DAFTAR TABEL **vi**

BAB I PENDAHULUAN **1**

1.1 Latar Belakang 1

1.2 Problem Statement 2

1.3 Tujuan 2

1.4 Target User dan Use Case 2

BAB II PERANCANGAN SISTEM **4**

2.1 Arsitektur Sistem 4

2.2 Daftar Komponen dan Alasan Pemilihan 5

2.3 Rangkaian dan Wiring 6

2.4 Pin dan GPIO 7

BAB III IMPLEMENTASI PROGRAM **9**

3.1 Logika Program 9

3.2 Komunikasi Serial 10

3.3 Komunikasi MQTT 10

3.4 Dashboard dan Visualisasi 11

BAB IV PENGUJIAN **13**

4.1 Skenario Pengujian 13

4.2 Screenshot dan Output 15

BAB V PEMBAGIAN PERAN ANGGOTA **17**

BAB VI PENUTUP **18**

6.1 Kesimpulan 18

6.2 Pengembangan Lanjutan 18

LAMPIRAN **19**

DAFTAR GAMBAR

Gambar 2.1 Block diagram sistem deteksi kebocoran gas LPG 5
Gambar 2.2 Rangkaian sistem pada simulator Wokwi 7

Gambar 3.1 Dashboard Google Colab saat kebocoran gas terdeteksi . . . 12

Gambar 4.1 TC1: kondisi normal dengan kadar gas 10 ppm 15
Gambar 4.2 TC2: gas terdeteksi (457 ppm) tanpa orang 15
Gambar 4.3 TC3: gas terdeteksi dan PIR mendeteksi gerakan 16
Gambar 4.4 TC4: tombol reset ditekan dan sistem kembali siaga 16

DAFTAR TABEL

Tabel 2.1 Arsitektur sistem per lapisan IoT 4
Tabel 2.2 Daftar komponen dan alasan pemilihan 6
Tabel 2.3 Penggunaan pin ESP32 8

Tabel 3.1 Pesan Serial Monitor 10
Tabel 3.2 Topic dan payload MQTT 11

Tabel 4.1 Skenario dan hasil pengujian 14

Tabel 5.1 Pembagian peran anggota 17

BAB I
PENDAHULUAN

1.1 Latar Belakang

Liquefied Petroleum Gas (LPG) atau gas elpiji adalah bahan bakar memasak yang dipakai luas di rumah tangga dan usaha kuliner di Indonesia, terutama sejak program konversi minyak tanah ke LPG yang dimulai pemerintah pada 2007. Kebocoran pada tabung, regulator, atau selang dapat memicu kebakaran dan ledakan jika gas yang bocor bertemu sumber api.

LPG lebih berat daripada udara, sehingga gas yang bocor cenderung mengumpul di dekat lantai dan tidak cepat hilang. Produsen menambahkan zat pembau agar kebocoran tercium, tetapi cara ini bergantung pada penghuni yang sedang berada di lokasi, terjaga, dan menyadari baunya. Saat penghuni tidur, berada di ruangan lain, atau tidak di rumah, kebocoran dapat berlangsung tanpa diketahui. Selain itu, regulator LPG rumah tangga yang umum dipakai hanya memiliki tuas manual, sehingga aliran gas baru tertutup jika ada orang yang memutarnya.

Dari sisi Keselamatan dan Kesehatan Kerja (K3), tindakan yang benar saat terjadi kebocoran adalah menutup aliran gas dari sumbernya dengan memutar tuas regulator ke posisi OFF. Mencabut selang dari regulator saat tuas masih ON justru berbahaya, karena gas dari tabung akan keluar bebas ke ruangan. Tindakan ini juga harus dilakukan tanpa menimbulkan percikan api, misalnya tanpa menyalakan kompor atau sakelar listrik.

Proyek ini merancang sistem berbasis Internet of Things (IoT) yang menjalankan langkah K3 tersebut secara otomatis: mendeteksi kebocoran gas, memutar tuas regulator ke posisi OFF menggunakan motor stepper, memberi peringatan suara kepada orang di sekitar lokasi, dan mengirim status ke dashboard agar kondisi dapat dipantau dari jarak jauh. Penggerak tuas dirancang untuk dipasang di luar regulator (non-invasif), sehingga bagian kedap gas regulator tidak perlu dibongkar. Seluruh rangkaian dirancang dan diuji pada simulator Wokwi menggunakan mikrokontroler ESP32.

1.2 Problem Statement

1. Bagaimana mendeteksi kebocoran gas LPG secara otomatis tanpa bergantung pada indra penciuman penghuni?
2. Bagaimana sistem menutup aliran gas dari sumbernya secara otomatis dengan cara yang aman menurut prinsip K3, sekaligus memperingatkan orang di sekitar lokasi?
3. Bagaimana memastikan alarm tidak mati dengan sendirinya sebelum kondisi diperiksa oleh manusia?
4. Bagaimana status sistem dapat dipantau dari jarak jauh?

1.3 Tujuan

1. Membaca sensor gas MQ-2 dengan ESP32 untuk mendeteksi kebocoran gas.
2. Menggerakkan motor stepper untuk memutar tuas regulator LPG dari posisi ON ke OFF saat kebocoran terdeteksi, dan membunyikan buzzer ketika sensor PIR mendeteksi keberadaan orang.
3. Menerapkan alarm terkunci (*latching*) yang hanya dapat dimatikan dengan tombol reset.
4. Mengirim data sensor dan status alarm melalui MQTT ke broker publik EMQX, lalu menampilkannya pada dashboard Python.

1.4 Target User dan Use Case

Target pengguna sistem ini adalah:

- Rumah tangga yang memakai kompor LPG, terutama yang dapurnya sering ditinggal atau dihuni lansia yang kepekaan penciumannya sudah menurun.
- Usaha kuliner kecil seperti warung makan yang memakai beberapa tabung LPG setiap hari.
- Pengelola kos atau asrama dengan dapur bersama yang tidak selalu diawasi.

Skenario penggunaan: saat terjadi kebocoran di dapur, sensor gas mendeteksinya dan sistem langsung mengunci alarm. Motor stepper memutar tuas regulator ke posisi OFF sehingga aliran gas dari tabung terhenti. Jika ada orang yang masuk ke area dapur, sensor PIR mendeteksinya dan buzzer berbunyi sebagai peringatan

agar orang tersebut tidak menyalakan api atau sakelar listrik. Pada saat yang sama, status kebocoran terkirim ke dashboard sehingga pemilik dapat memantaunya dari tempat lain. Setelah lokasi diperiksa dan dinyatakan aman, pengguna menekan tombol reset; motor memutar tuas kembali ke posisi ON dan sistem kembali siaga.

BAB II
PERANCANGAN SISTEM

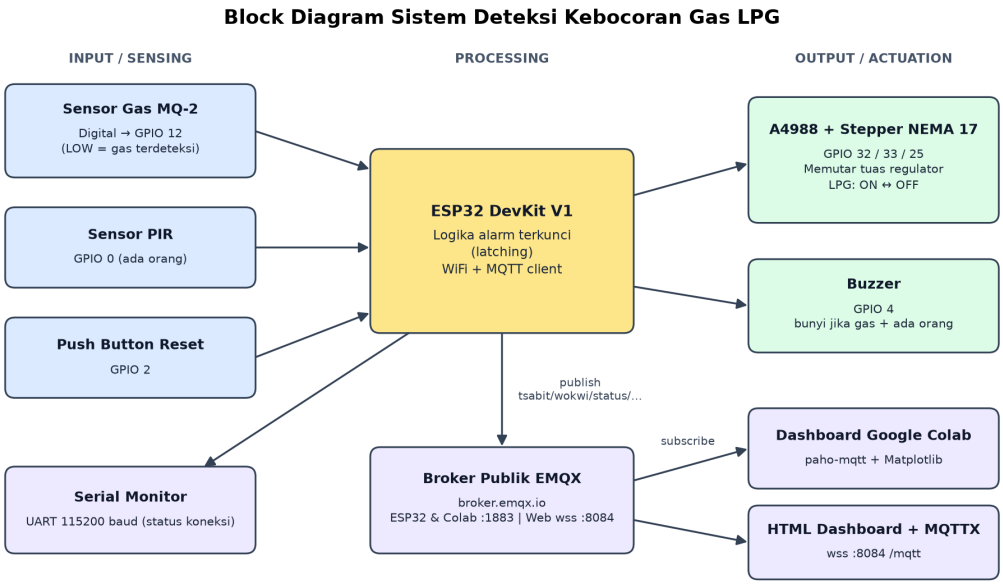
2.1 Arsitektur Sistem

Sistem mengikuti alur IoT: *sensing*, *processing*, *actuation*, *communication*, dan *visualization*. Peran setiap lapisan dirangkum pada Tabel 2.1.

Tabel 2.1 Arsitektur sistem per lapisan IoT

Lapisan	**Komponen**	**Peran**
Input / Sensing	Sensor gas MQ-2, sensor PIR, push button	Mendeteksi gas, keberadaan orang, dan perintah reset
Processing / Logic	ESP32 DevKit V1	Menjalankan logika alarm terkunci dan mengendalikan aktuator
Output / Actuation	Driver A4988 dan motor stepper NEMA 17, buzzer	Menggerakkan aktuator dan memberi peringatan suara
Communication	Serial Monitor, WiFi, MQTT (broker publik EMQX)	Mengirim log debugging dan data status
Visualization	Google Colab (paho-mqtt dan Matplotlib) dan dashboard web	Menampilkan status sensor dan aktuator secara live

Data mengalir dari sensor gas, sensor PIR, dan tombol ke ESP32. ESP32 memutuskan respons dan mengendalikan motor stepper serta buzzer. Secara bersamaan, ESP32 mengirim log ke Serial Monitor dan data status melalui MQTT ke broker publik EMQX (`broker.emqx.io`), lalu dashboard Python di Google Colab dan dashboard web membaca data tersebut dari broker. Alur ini digambarkan pada Gambar 2.1.



Gambar 2.1 Block diagram sistem deteksi kebocoran gas LPG

2.2 Daftar Komponen dan Alasan Pemilihan

Komponen dipilih berdasarkan kebutuhan use case dan ketersediaannya di Wokwi, seperti terlihat pada Tabel 2.2.

Tabel 2.2 Daftar komponen dan alasan pemilihan

Komponen	Fungsi	Alasan Pemilihan
ESP32 DevKit V1	Controller utama	Ditetapkan oleh brief; memiliki WiFi bawaan untuk MQTT
Sensor gas MQ-2	Mendeteksi kebocoran gas	Peka terhadap gas mudah terbakar termasuk LPG; output digitalnya langsung menunjukkan ada atau tidaknya gas berdasarkan ambang yang diatur di modul
Sensor PIR	Mendeteksi keberadaan orang	Mendeteksi panas tubuh yang bergerak, sehingga peringatan hanya dibunyikan ketika ada orang di lokasi
Motor stepper NEMA 17	Pemutar tuas regulator ON/OFF	Posisinya dapat dihitung per langkah, sehingga motor kembali tepat ke posisi awal tanpa sensor posisi
Driver A4988	Penggerak motor stepper	Pin ESP32 tidak mampu mengalirkan arus koil motor; A4988 cukup dikendalikan dengan sinyal STEP dan DIR
Buzzer	Peringatan suara	Indikator sederhana untuk memperingatkan orang di sekitar lokasi
Push button	Reset alarm	Memastikan alarm hanya dimatikan oleh manusia setelah kondisi diperiksa

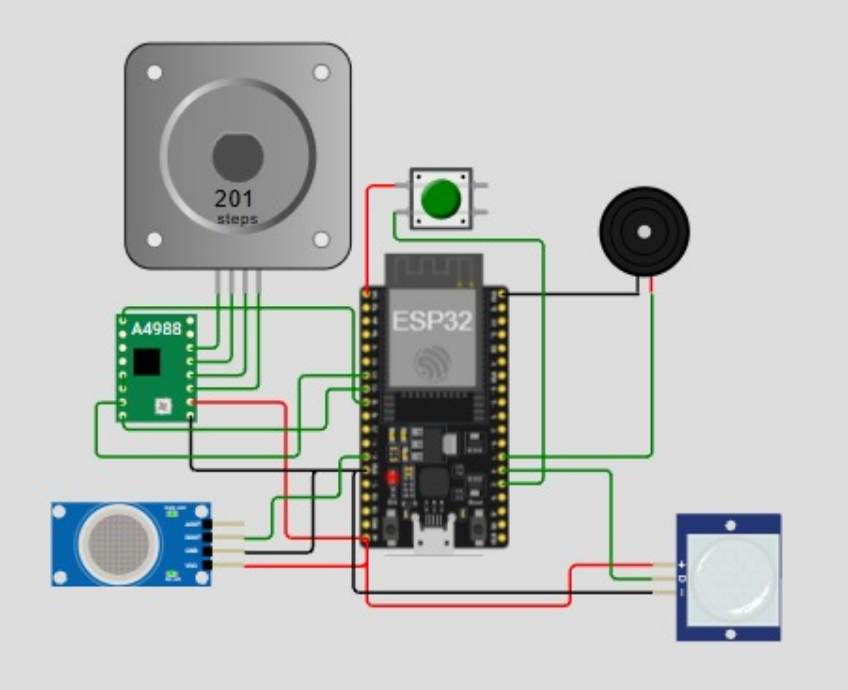
2.3 Rangkaian dan Wiring

Rangkaian dibuat di Wokwi (Gambar 2.2). Sensor gas dan sensor PIR mendapat daya 3V3 dari ESP32. Seluruh ground terhubung ke GND ESP32 agar semua sinyal diukur terhadap titik nol yang sama. Hal penting pada wiring:

- Pin ENABLE A4988 dikendalikan GPIO 25 dan bersifat aktif-LOW: LOW menghidupkan driver, HIGH mematikannya agar motor tidak panas saat diam.
- Driver A4988 hanya dikabel pada pin kendali (STEP, DIR, ENABLE) dan keempat output koil. Catu daya motor (VMOT) tidak diperlukan pada simulasi Wokwi, tetapi wajib pada perangkat keras nyata.
- Koil motor terhubung ke pasangan output 1A/1B untuk koil A dan 2A/2B untuk koil B.
- Tombol reset menghubungkan GPIO 2 ke GND, dan program mengaktifkan resistor pull-down internal pada pin tersebut. Program menganggap tombol

ditekan ketika pin terbaca HIGH.

- Sensor gas memakai output digital yang terhubung ke GPIO 12. Modul bersifat aktif-LOW, sehingga LOW berarti gas terdeteksi.



Gambar 2.2 Rangkaian sistem pada simulator Wokwi

Link project Wokwi tercantum pada Lampiran.

2.4 Pin dan GPIO

Penggunaan pin ESP32 dirangkum pada Tabel 2.3.

Tabel 2.3 Penggunaan pin ESP32

Pin ESP32	**Terhubung ke**	**Mode**	**Fungsi**
GPIO 32	A4988 STEP	OUTPUT	Pulsa langkah motor
GPIO 33	A4988 DIR	OUTPUT	Arah putaran motor
GPIO 25	A4988 ENABLE	OUTPUT	Menghidupkan atau mematikan driver (aktif-LOW)
GPIO 12	MQ-2 (output digital)	INPUT	Deteksi gas (LOW berarti gas terdeteksi)
GPIO 0	PIR OUT	INPUT	Deteksi keberadaan orang
GPIO 2	Push button	INPUT_PULLDOWN	Reset alarm
GPIO 4	Buzzer	OUTPUT	Peringatan suara
TX / RX	Serial Monitor	UART 115200 baud	Status koneksi WiFi dan MQTT

BAB III
IMPLEMENTASI PROGRAM

3.1 Logika Program

Program ditulis dalam bahasa C++ (Arduino) untuk ESP32 dengan alur kerja sebagai berikut:

1. Saat menyala, ESP32 mengatur mode setiap pin, mematikan driver motor dan buzzer, lalu menunggu sampai WiFi tersambung dan mengatur alamat broker MQTT.
2. Pada setiap putaran loop, ESP32 memastikan koneksi MQTT aktif, membaca sensor gas, sensor PIR, dan tombol reset, lalu mengirim status yang berubah ke broker. Modul MQ-2 bersifat aktif-LOW, sehingga nilai LOW berarti gas terdeteksi.
3. Jika gas terdeteksi, program masuk ke blok alarm. Blok ini hanya dapat ditinggalkan dengan menekan tombol reset, sehingga alarm tetap aktif walaupun kadar gas sudah turun.
4. Di dalam blok alarm, motor stepper bergerak 200 langkah (satu putaran pada mode full-step) satu kali untuk memutar tuas regulator ke posisi OFF, dan posisinya dicatat dalam variabel `count`. Pada simulasi, satu putaran ini merepresentasikan gerakan tuas dari ON ke OFF; rasio gir dan sudut tuas sebenarnya ditentukan saat prototipe fisik dibuat.
5. Selama alarm aktif, program membaca sensor PIR. Buzzer berbunyi ketika ada orang dan mati ketika tidak ada orang.
6. Saat tombol reset ditekan, buzzer dimatikan dan motor bergerak mundur 200 langkah, sehingga tuas kembali ke posisi ON. Setelah itu sistem kembali siaga.

Alarm dibuat terkunci karena kadar gas dapat turun sesaat, misalnya karena tertiup angin, padahal sumber kebocoran belum ditangani. Dengan alarm terkunci, sistem baru kembali normal setelah manusia memeriksa lokasi dan menekan tombol reset. Kode program lengkap tercantum pada Lampiran.

3.2 Komunikasi Serial

Serial Monitor (115200 baud) dipakai untuk debugging. Serial Monitor menampilkan proses koneksi ke WiFi dan broker MQTT, sedangkan perubahan status sensor dan aktuator dipantau melalui MQTT. Daftar pesan tercantum pada Tabel 3.1.

Tabel 3.1 Pesan Serial Monitor

Pesan	**Kondisi**
`[WiFi] Connecting to Wokwi-GUEST...`	ESP32 mulai menyambung ke WiFi
`[WiFi] Connected!`	WiFi tersambung
`[MQTT] Connecting to broker.emqx.io... CONNECTED!`	Koneksi ke broker publik EMQX berhasil

3.3 Komunikasi MQTT

ESP32 terhubung ke WiFi, lalu ke broker publik EMQX (`broker.emqx.io`) melalui MQTT pada port 1883. Dashboard Colab memakai port yang sama, sedangkan dashboard web di browser memakai MQTT over WebSocket aman (wss) pada port 8084 dengan path `/mqtt`, karena browser tidak dapat membuka koneksi TCP langsung seperti ESP32. Broker ini dipakai karena paket gratis HiveMQ Cloud sudah tidak tersedia untuk pengguna baru; broker publik EMQX gratis, tidak memerlukan akun, dan tetap memakai protokol MQTT standar, sehingga penguji dapat menjalankan simulasi dan dashboard tanpa kredensial.

Setiap komponen memiliki topic sendiri di bawah awalan `tsabit/wokwi/status/` dengan payload berupa teks. Status dikirim dengan dua cara: segera ketika nilainya berubah, dan secara rutin setiap 1 detik agar dashboard yang baru terhubung langsung mendapat kondisi terkini. Pesan dikirim dengan QoS 0 tanpa retain. Topic dan payload yang dipakai tercantum pada Tabel 3.2.

Tabel 3.2 Topic dan payload MQTT

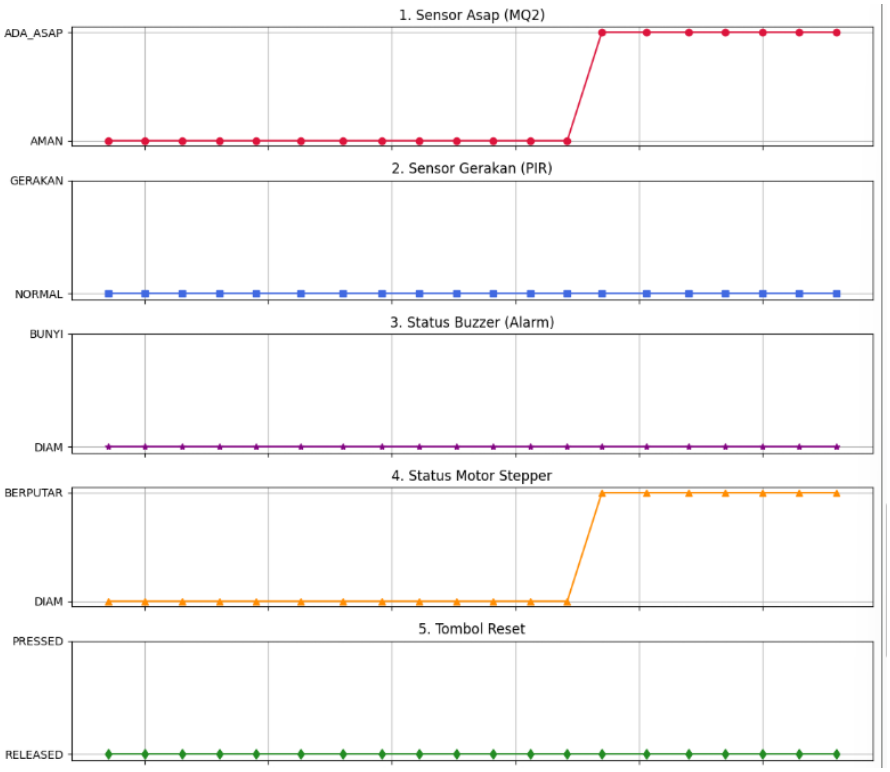
Topic	Payload	Arti
`tsabit/wokwi/status/asap`	`ADA_ASAP` atau `AMAN`	Gas terdeteksi atau udara normal
`tsabit/wokwi/status/pir`	`GERAKAN_DETECTED` atau `NORMAL`	Ada gerakan orang atau area kosong
`tsabit/wokwi/status/buzzer`	`BUNYI` atau `DIAM`	Status buzzer peringatan
`tsabit/wokwi/status/motor`	`BERPUTAR` atau `DIAM`	Status motor penggerak tuas regulator
`tsabit/wokwi/status/button`	`PRESSED` atau `RELEASED`	Status tombol reset

Nama topic `asap` dan payload `ADA_ASAP` mengikuti penamaan di program; yang dideteksi sistem adalah kebocoran gas LPG. Karena payload berupa teks biasa, pesan dapat dibaca langsung di aplikasi MQTTX tanpa perlu diolah.

3.4 Dashboard dan Visualisasi

Dashboard utama dibuat di Google Colab menggunakan pustaka paho-mqtt untuk menerima data dan Matplotlib untuk visualisasi. Notebook berlangganan ke topic `tsabit/wokwi/#` dan memperbarui tampilan setiap 1 detik, dengan isi sebagai berikut:

- Lima grafik status yang masing-masing bernilai 0 atau 1: sensor gas, sensor PIR, buzzer, motor stepper, dan tombol reset.
- Sumbu waktu dalam detik yang menampilkan 20 titik data terakhir, sehingga urutan perubahan status terlihat jelas.
- Sebagai pelengkap, dashboard web berbasis HTML dan JavaScript menampilkan kelima status sebagai kartu berwarna yang berubah secara real-time melalui WebSocket.
- Pesan juga dapat dipantau langsung dengan aplikasi MQTTX dengan berlangganan ke topic yang sama.



Gambar 3.1 Dashboard Google Colab saat kebocoran gas terdeteksi

BAB IV
PENGUJIAN

4.1 Skenario Pengujian

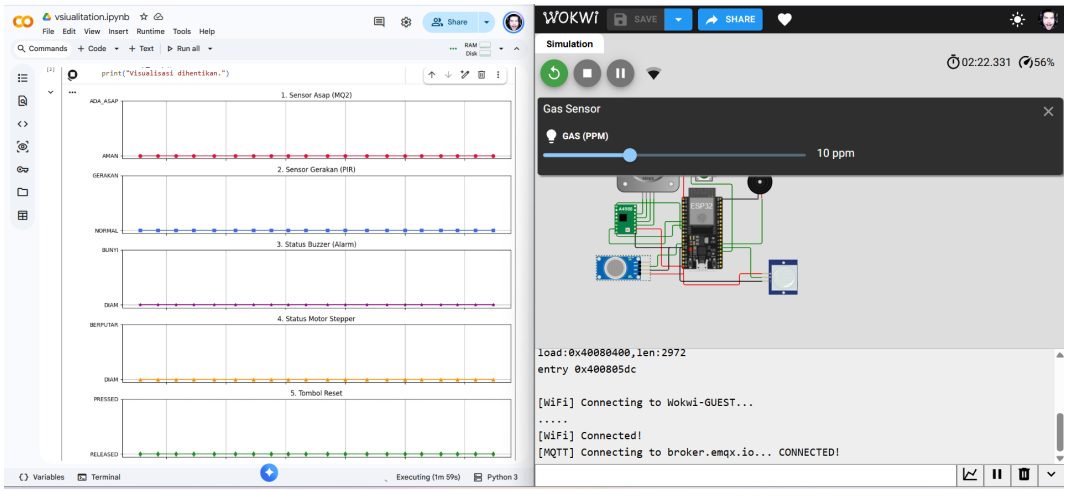
Pengujian dilakukan pada simulator Wokwi dengan dashboard Colab berjalan bersamaan. Pengujian mencakup kondisi normal (TC1) dan kondisi abnormal (TC2 sampai TC4). Skenario dan hasilnya tercatat pada Tabel 4.1, dengan bukti berupa screenshot pada subbab berikutnya.

Tabel 4.1 Skenario dan hasil pengujian

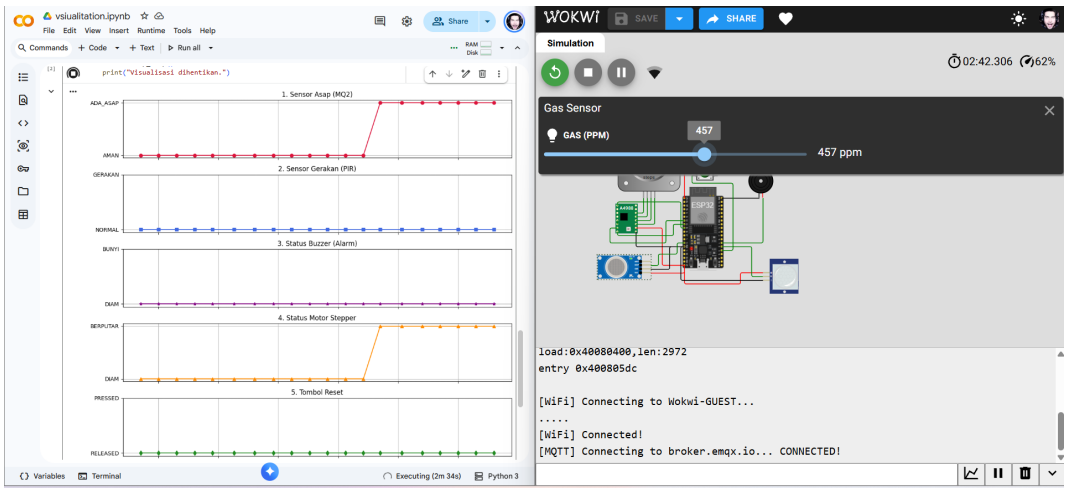
ID	Skenario	Input / Kondisi	Hasil yang Diharapkan	Hasil Aktual	Status
TC1	Kondisi normal	Gas rendah, tidak ada gerakan	Buzzer dan motor diam; dashboard menampilkan AMAN, NORMAL, DIAM, DIAM, dan RELEASED	Dengan kadar gas 10 ppm, motor dan buzzer diam; kelima grafik dashboard tetap pada AMAN, NORMAL, DIAM, DIAM, dan RELEASED (Gambar 4.1)	Berhasil
TC2	Gas terdeteksi tanpa orang	Kadar gas dinaikkan di Wokwi, PIR diam	Alarm terkunci, motor bergerak 200 langkah, buzzer diam; dashboard menampilkan ADA_ASAP dan motor BERPUTAR	Saat kadar gas dinaikkan ke 457 ppm, dashboard berubah ke ADA_ASAP dan motor BERPUTAR, sedangkan buzzer tetap DIAM karena PIR NORMAL (Gambar 4.2)	Berhasil
TC3	Gas terdeteksi dan ada orang	Saat alarm aktif, PIR dipicu	Buzzer berbunyi; dashboard menampilkan GERAKAN_ DETECTED dan BUNYI	Setelah gerakan disimulasikan, dashboard menampilkan GERAKAN dan buzzer BUNYI; penghitung langkah stepper di Wokwi menunjukkan 201 (Gambar 4.3)	Berhasil
TC4	Reset	Tombol reset ditekan saat alarm aktif	Buzzer mati, motor mundur 200 langkah; dashboard menampilkan PRESSED, lalu motor DIAM	Setelah gas diturunkan dan tombol ditekan, dashboard menampilkan PRESSED sesaat, lalu motor DIAM; penghitung langkah turun dari 201 ke 1, artinya motor mundur 200 langkah (Gambar 4.4)	Berhasil

4.2 Screenshot dan Output

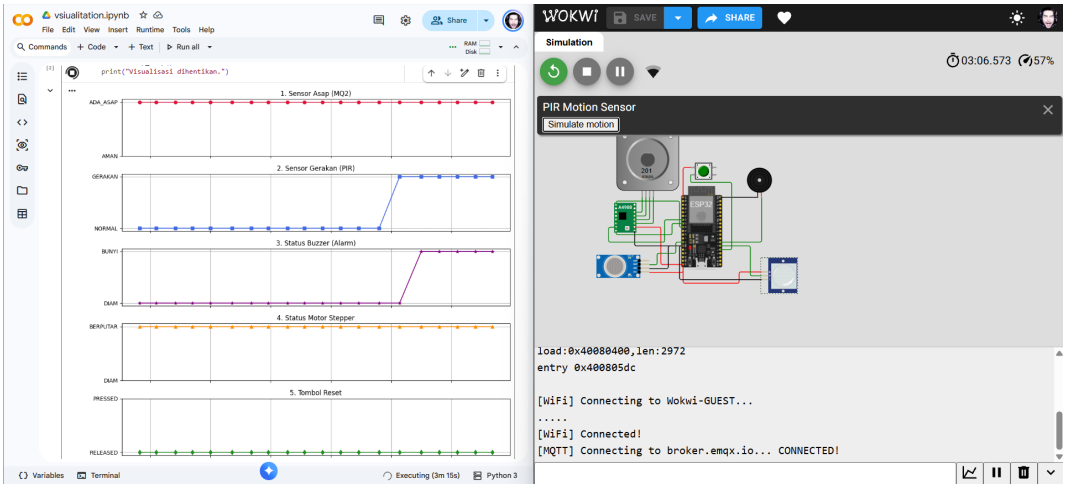
Setiap screenshot memperlihatkan dashboard Google Colab (kiri) serta rangkaian dan Serial Monitor Wokwi (kanan) pada saat yang sama. Serial Monitor pada semua gambar menunjukkan ESP32 berhasil terhubung ke WiFi dan broker MQTT.



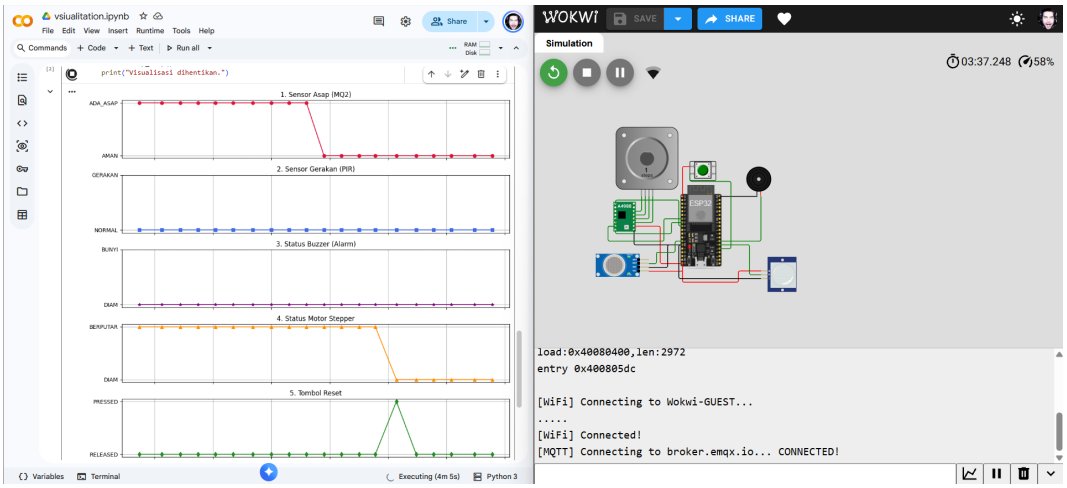
Gambar 4.1 TC1: kondisi normal dengan kadar gas 10 ppm



Gambar 4.2 TC2: gas terdeteksi (457 ppm) tanpa orang



Gambar 4.3 TC3: gas terdeteksi dan PIR mendeteksi gerakan



Gambar 4.4 TC4: tombol reset ditekan dan sistem kembali siaga

BAB V
PEMBAGIAN PERAN ANGGOTA

Proyek dikerjakan oleh empat anggota. Setiap anggota memegang satu bagian utama dari alur IoT, yaitu perancangan rangkaian dan pemrograman, simulasi dan pengujian, komunikasi dan dokumentasi, serta visualisasi data. Dengan pembagian ini, setiap tahap dalam alur sistem memiliki penanggung jawab yang jelas. Rincian peran dan kontribusi setiap anggota tercantum pada Tabel 5.1.

Tabel 5.1 Pembagian peran anggota

Nama Anggota	**Peran**	**Kontribusi**
Frenaldy Bestabba Hasugian	Dokumentasi dan Komunikasi MQTT	Menyusun laporan proyek (Bab I sampai VI, block diagram, dan lampiran). Mengatur MQTTX Web untuk memantau seluruh topic status pada broker EMQX, sekaligus memastikan pesan dari ESP32 benar-benar terkirim.
Muhammad Sayyid Tsabit	Simulasi Wokwi dan Pengujian	Menyiapkan dan menjalankan proyek simulasi di Wokwi. Menjalankan skenario pengujian TC1 sampai TC4, lalu mendokumentasikan hasilnya dalam bentuk screenshot (Gambar 4.1 sampai 4.4).
Nicholas	Perancangan Rangkaian dan Pemrograman ESP32	Merancang rangkaian pada `diagram.json`, yang terdiri dari ESP32, sensor gas, PIR, driver A4988, motor stepper, buzzer, dan tombol reset. Menulis program `sketch.ino`, termasuk logika alarm terkunci, kendali motor stepper, dan pengiriman status melalui MQTT.
Mitra Partogi	Dashboard dan Visualisasi Data	Membuat dashboard Google Colab dengan paho-mqtt dan Matplotlib, serta dashboard HTML. Keduanya menampilkan status gas, PIR, buzzer, motor, dan tombol secara real-time (Gambar 3.1).

BAB VI
PENUTUP

6.1 Kesimpulan

Sistem deteksi kebocoran gas LPG berbasis ESP32 dirancang dengan alur IoT lengkap, mulai dari *sensing* dengan sensor gas MQ-2 dan PIR, logika alarm terkunci pada ESP32, aktuasi berupa motor stepper yang memutar tuas regulator ke posisi OFF dan buzzer peringatan, komunikasi melalui Serial dan MQTT ke broker publik EMQX, hingga visualisasi pada dashboard Python. Alarm terkunci memastikan sistem tidak kembali normal sebelum lokasi diperiksa. Pendekatan ini mengikuti prinsip K3: aliran gas ditutup dari sumbernya tanpa mencabut selang dan tanpa membongkar regulator.

Keempat skenario pengujian (TC1 sampai TC4) berhasil. Sistem tetap siaga saat udara normal, menutup tuas regulator saat gas terdeteksi, membunyikan buzzer hanya ketika ada orang di lokasi, dan kembali siaga setelah tombol reset ditekan. Seluruh perubahan status juga tampil pada dashboard melalui MQTT secara real-time.

6.2 Pengembangan Lanjutan

- Membuat dudukan dan gir penghubung stepper ke tuas regulator pada prototipe fisik, lalu menyesuaikan jumlah langkah motor dengan sudut putar tuas dan torsi yang dibutuhkan.
- Mengirim notifikasi ke ponsel, misalnya melalui Telegram, ketika alarm aktif.
- Membaca output analog sensor MQ-2 dan mengkalibrasinya ke satuan ppm, sehingga dashboard dapat menampilkan tren kadar gas dan peringatan dini sebelum ambang tercapai.
- Memindahkan sensor dari strapping pin ESP32 (GPIO 0 dan 12) dan menambahkan level shifter sebelum diterapkan pada perangkat keras nyata.
- Mengubah program menjadi non-blocking dan membuat alarm tetap bekerja tanpa koneksi WiFi atau broker, serta memakai broker pribadi dengan autentikasi dan TLS untuk koneksi ESP32 pada produk nyata.

LAMPIRAN

A. Tautan Proyek

- Link project Wokwi:
  `https://wokwi.com/projects/479232165870904321`
- Source code:
  `https://github.com/FrenaldyH/samsung-iot-project`
- Video demo:
  `https://youtu.be/zDZDpb93hAY`

B. Kode Program ESP32 (sketch.ino)

```
#include <WiFi.h>
#include <PubSubClient.h>

// --- Pin Definitions ---
#define Step 32
#define DIR 33
#define MQ2 12
#define PIR 0
#define Reset 2
#define Buzz 4
#define EN 25

// --- Network & Broker Configuration ---
const char* ssid        = "Wokwi-GUEST";
const char* password    = "";
const char* mqtt_server = "broker.emqx.io";
const int   mqtt_port   = 1883;

// --- MQTT Topics Specification ---
const char* topic_asap   = "tsabit/wokwi/status/asap";
const char* topic_pir    = "tsabit/wokwi/status/pir";
const char* topic_buzzer = "tsabit/wokwi/status/buzzer";
const char* topic_motor  = "tsabit/wokwi/status/motor";
const char* topic_button = "tsabit/wokwi/status/button";

WiFiClient espClient;
PubSubClient client(espClient);

// --- Global Variables & State Tracking ---
int count = 0;
bool asap = HIGH;
```

```
bool lastAsap = -1;

bool pirVal = LOW;
bool lastPirVal = -1;

bool buzzVal = LOW;
bool lastBuzzVal = -1;

bool butsig = LOW;
bool lastButsig = -1;

String motorStatus = "DIAM";
String lastMotorStatus = "";

unsigned long lastMqttSend = 0;

void setup_wifi() {
  delay(10);
  Serial.println("\n[WiFi] Connecting to Wokwi-GUEST...");
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\n[WiFi] Connected!");
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("[MQTT] Connecting to broker.emqx.io...");
    String clientId = "ESP32-Wokwi-";
    clientId += String(random(0xffff), HEX);
    if (client.connect(clientId.c_str())) {
      Serial.println(" CONNECTED!");
    } else {
      delay(5000);
    }
  }
}

void checkAndPublishAll() {
  // 1. Asap (MQ2)
  asap = digitalRead(MQ2);
  if (asap != lastAsap) {
    lastAsap = asap;
    client.publish(topic_asap, (asap == LOW) ? "ADA_ASAP" : "AMAN");
  }

  // 2. PIR Sensor
  pirVal = digitalRead(PIR);
  if (pirVal != lastPirVal) {
    lastPirVal = pirVal;
```

```
    client.publish(topic_pir, (pirVal == HIGH) ? "GERAKAN_DETECTED" : "
    NORMAL");
  }

  // 3. Buzzer
  if (buzzVal != lastBuzzVal) {
    lastBuzzVal = buzzVal;
    client.publish(topic_buzzer, (buzzVal == HIGH) ? "BUNYI" : "DIAM");
  }

  // 4. Tombol Reset
  butsig = digitalRead(Reset);
  if (butsig != lastButsig) {
    lastButsig = butsig;
    client.publish(topic_button, (butsig == HIGH) ? "PRESSED" : "RELEASED")
    ;
  }

  // 5. Motor Stepper
  if (motorStatus != lastMotorStatus) {
    lastMotorStatus = motorStatus;
    client.publish(topic_motor, motorStatus.c_str());
  }
}

void sendRutin() {
  if (millis() - lastMqttSend > 1000) {
    lastMqttSend = millis();
    client.publish(topic_asap, (asap == LOW) ? "ADA_ASAP" : "AMAN");
    client.publish(topic_pir, (pirVal == HIGH) ? "GERAKAN_DETECTED" : "
    NORMAL");
    client.publish(topic_buzzer, (buzzVal == HIGH) ? "BUNYI" : "DIAM");
    client.publish(topic_motor, motorStatus.c_str());
    client.publish(topic_button, (butsig == HIGH) ? "PRESSED" : "RELEASED")
    ;
  }
}

void setup() {
  Serial.begin(115200);

  pinMode(Step, OUTPUT);
  pinMode(DIR, OUTPUT);
  pinMode(Buzz, OUTPUT);
  pinMode(Reset, INPUT_PULLDOWN);
  pinMode(PIR, INPUT);
  pinMode(MQ2, INPUT);
  pinMode(EN, OUTPUT);

  digitalWrite(EN, HIGH);
  digitalWrite(Buzz, LOW);

  setup_wifi();
```

```
  client.setServer(mqtt_server, mqtt_port);
}

void loop() {
  if (!client.connected()) reconnect();
  client.loop();

  checkAndPublishAll();
  sendRutin();

  // --- Logika Darurat Kebocoran Gas ---
  if (asap == LOW) {
    while (butsig == LOW) {
      client.loop();
      checkAndPublishAll();
      sendRutin();

      butsig = digitalRead(Reset);

      if (pirVal == HIGH) {
        digitalWrite(Buzz, HIGH);
        buzzVal = HIGH;
      } else {
        digitalWrite(Buzz, LOW);
        buzzVal = LOW;
      }

      // Memutar Stepper Motor memutus Knob Regulator Winn Gas W-688 M (Posisi ON -> OFF)
      if (count != 200) {
        motorStatus = "BERPUTAR";
        checkAndPublishAll();

        digitalWrite(EN, LOW);
        digitalWrite(DIR, HIGH);
        for (int i = 0; i < 200; i++) {
          digitalWrite(Step, HIGH);
          delayMicroseconds(1000);
          digitalWrite(Step, LOW);
          delayMicroseconds(1000);
          count += 1;
        }
        digitalWrite(EN, HIGH);
      }
    }

    // --- Eksekusi Tombol Reset (Memutar Balik Knob Regulator OFF -> ON) ---
    motorStatus = "BERPUTAR";
    buzzVal = LOW;
    digitalWrite(Buzz, LOW);
    checkAndPublishAll();

```

```
    digitalWrite(EN, LOW);
    digitalWrite(DIR, LOW);

    for (int i = 0; i < 200; i++) {
      digitalWrite(Step, HIGH);
      delayMicroseconds(1000);
      digitalWrite(Step, LOW);
      delayMicroseconds(1000);
      count -= 1;
    }

    digitalWrite(EN, HIGH);
    motorStatus = "DIAM";
    checkAndPublishAll();
    delay(500);
  } else {
    buzzVal = LOW;
    digitalWrite(Buzz, LOW);
    motorStatus = "DIAM";
  }
}
```